コンピュータサイエンス第一 第一回課題解答解説 v2

今回のポイント

- 変数の定義と参照
- 数値・文字列の扱い
- Python における加算・結合演算子(「+」記号)

各問の解答解説

問 1

```
a=1
b=2
c=3
print(a + b + c)
```

これによって出力されるのは **6** です。プログラムの流れとしては、まず a,b,c の 3 つの変数を定義し、それらに初期値として整数型(int 型)である 1,2,3 を設定しています。そして print 関数の引数に `a + b + c` という式を渡しています。これは整数型同士を「+」で繋いでいるので、加算演算子(足し算して)と解釈され、最終的な出力として **6** が出ます。

細かい解説

> 整数型同士の加算の場合は式に変数の中身をそのまま置き換える、というイメージでオッケーです。今回の場合は `a=1 b=2 c=3` ですから、`a + b + c` は単純に `1 + 2 + 3` となるわけですね。

問 2

```
a="1"
b="2"
c="3"
print(a + b + c)
```

これによって出力されるのは **123** です。プログラムの流れはさっきのと似ていますが、最初の変数定義の初期値が異なります。まず a,b,c の 3 つの変数を定義するのは同じですが、それらに初期値として**整数型ではなく**文字列型(str 型)である"1","2","3"を設定しています。そして print 関数の引数に `a`

`+ b + c` を渡すところは同じです。しかし、今回は文字列型同士を「+」で繋いでいるので、**加算演算子ではなく**結合演算子（この文字列を繋げて）と解釈され、最終的な出力として **123** が出ます。

細かい解説

> 文字列型同士の結合でも一旦式に変数の中身を置き換えてみるとわかりやすいかもしれません。今回の場合は式は `"1" + "2" + "3"` となります。あとは単純に文字列同士をつなげてあげるだけなので、`"123"`、すなわち **123** が出力されるわけですね。

注. python では文字列型は主に`"`(ダブルクォーテーション) もしくは`'`(シングルクォーテーション) で囲んで表します。なんで囲むのか、なんで二種類あるのかはとりあえずそういうもんだと思っていてください。

問 3

```
a=1
b=2
c=3
print(str(a) + str(b) + str(c))
```

これによって出力されるのも **123** です。プログラムの流れはほぼ同じです。まず a,b,c の 3 つの変数を定義し、それらに初期値として int 型 (整数型) である 1,2,3 を設定しています。しかし異なるところとして print 関数の引数に `str(a) + str(b) + str(c)` を渡しています。`str()` 関数は引数に指定された (ほぼ) あらゆるものを文字列型に変換してくれます。つまり形が違うだけで問 2 と同じことをさせています。これは文字列型同士を「+」で繋いでいるので、さっきと同じく結合演算子と解釈され、最終的な出力として **123** が出ます。

注. python には様々な型があります。それらを (いい感じに) 変換して使うために、python には `str()` 以外にも様々な型変換関数が存在します。例えば `int()` は整数に、`float()` は浮動小数点数に、という具合に。ただこれらは割と無理やりな変換をするときもあるので (小数から整数への変換のときに切り捨てされるとか)、そうされたくないときはちゃんと自分で色々処理を書いてなんとかしなきゃいけないわけですね。

問 4

```
a=1
b=2
c=3
print(a, b, c)
```

これによって出力されるのは **1 2 3** です。プログラムの流れとしては、まず a,b,c の 3 つの変数を定義し、それらに初期値として int 型 (整数型) である 1,2,3 を設定しています。そして print 関数の引数に `a, b, c` を渡しています。Python における print 関数の仕様として、カンマで区切って値を与えられると、スペースでその値同士を区切って表示するという性質があります。その仕様により最終的な出力として **1 2 3** が出ます。

細かい解説

> 今回は全部同じ整数型をカンマで区切って渡しましたが、`print(1, 3.14, "Hello")` のように違う型の値同士を区切って渡してもちゃんと表示してくれます。

問 5

```
x=10
y=5
print(x + y + z)
```

これによって出力されるのは**エラー**です。プログラムの流れとしては、まず x,y の 2 つの変数を定義し、それらに初期値として int 型 (整数型) である 10,5 を設定しています。そして print 関数の引数に `x + y + z` を渡しています。あれ、x,y はいいですが、z に関しては**事前に**定義していないのに使ってしまっています。python では変数は使う前に定義していなければならないルールがあります。これはそのルールに違反しているので、他の部分とかは何も関係無しに**エラー**が出て終了します。

注. エラーを恐れないでください。エラーはバグに立ち向かうあなたの味方です。間違ったところをだいたい教えてくれるんです。今回の例だと多分こういうエラーが出るんですが、

```
Traceback (most recent call last):
  File "/Users/yuto/ComputerScience1/tst.py", line 3, in <module>
    print(x + y + z)
                  ^
NameError: name 'z' is not defined
```

ほら、エラーの二行目において `line 3` って、プログラムの 3 行目だって教えてくれているんです。さらにエラーの三、四行目ではわざわざ矢印まで使って丁寧に場所を教えてくれます。で、最後の行で `NameError: name 'z' is not defined` と、違反したルールのことについても教えてくれてるんです。エラーは赤文字とかで出るから多少威圧感はあるでしょうが、基本的にあなたの味方です。たくさんエラーを出してバグを見つけましょう。

問 6

```
a=10
a=a+5
print(a)
```

これによって出力されるのは **15** です。プログラムの流れとしては、まず a という名前の変数を定義し、それに初期値として int 型 (整数型) である 10 を設定しています。その次の行では、多分プログラミング以外では一生成立しなさそうな等式が見えますね。しかしこれは等式ではありません。python では **=**(イコール 1 個) は代入演算子として解釈されます。つまり「左にある変数に右の値を入れろ」となるのです。つまり、今回の場合は変数 a に対してもともとの変数 a の中身である 10 と 5 を足して (加算演算子)、代入しろっていう意味となるわけです。そして print 関数の引数に **a** を渡しているので、最終的な出力として 1 行前で新しく a の中身となった **15** が出ます。

問 7

```
a=10
# a=a+5
print(a)
```

これによって出力されるのは **10** です。プログラムの流れは問 6 と全く同じに見えます。しかし、2 行目の行頭に **#** があります。これは python においては「この行のこのあとの内容はコメントにする」ということを表します。よってこの行は無視されるため、最初に変数 a に設定した値がそのままになり、最終的な出力として **10** が出ます。

細かい解説

コメントは、コンピュータ側からは完全に無視されるので、プログラムの意図や仕組み等を人間同士でわかりやすくするために使われています。また、例えば数十行、数百行ものコードが一旦要らなくなったからといって一気に消してしまうのは怖いから、とりあえず、暫定的な措置として全体をコメントにしてしまって、処理から消すということもあります。(コメントにしてしまえば内容がなんであってもコンピュータ、python からは無視されるので、手軽に実験ができるわけです) こういう行為を「コメントアウト」と呼びます。